

UNIVERSIDAD SALESIANA DE BOLIVIA



MANUAL TECNICO

ControlFuel

Sistema embebido con ESP32 para el control de acceso, medición de nivel de gasolina y racionamiento inteligente de combustible en tanques subterráneos de estaciones de servicio.

INTEGRANTES:

- Oliver Choque Pachacuti
- Danner Suxo Quispe
- Franklin Adhemar Pongo Cori
- Willka Daniel Condori Ventura
- Andrei Gabriel Rada Brun
- Alan Jhohan Del Rio Lopez

Materia: Ingeniería de Sistemas I

Semestre: 8vo Semestre

Paralelo: 8 - 111

LA PAZ – BOLIVIA

2026

ÍNDICE GENERAL

1. Proposito.....	3
2. Incremento técnico mínimo alcanzado	3
3. Alcance funcional	5
4. Requisitos	6
5. Instalación de la base de datos	6
6. Observación importante sobre la configuración.....	7
7. Configuración del backend.....	7
8. Crear el usuario administrador.....	8
9. Ejecución del sistema web.....	8
10.Ejecución del hardware.....	8
11.Endpoints HTTP del backend revisado.....	9
12.Estructura recomendada del repositorio.....	10
13.Seguridad y alcance académico.....	10
14.Estado de validación.....	11

ControlFuel

Sistema embebido y plataforma web para el control de acceso, medición del nivel de combustible y racionamiento inteligente en un tanque de prueba.

Integración de hardware y software en una prueba de concepto académica. El sistema combina un ESP32, lector RFID RC522, sensor ultrasónico, actuador de corte y una plataforma web PHP/MySQL.

1. Propósito

ControlFuel busca reemplazar parte del control manual de despacho por un flujo digital trazable:

1. el operador presenta una tarjeta RFID;
2. el sistema valida la credencial;
3. el sensor ultrasónico reporta el nivel estimado;
4. el servidor aplica las reglas de modo normal o modo crisis;
5. el ESP32 habilita o mantiene bloqueado el actuador;
6. el evento queda guardado en MySQL y se muestra en el dashboard.

El prototipo se prueba a escala de laboratorio. No está certificado para atmósferas explosivas, estaciones de servicio reales ni conexión con ANH/B-SISA.

2. Incremento técnico mínimo alcanzado

El incremento técnico de esta etapa es una versión demostrable de extremo a extremo compuesta por:

Hardware

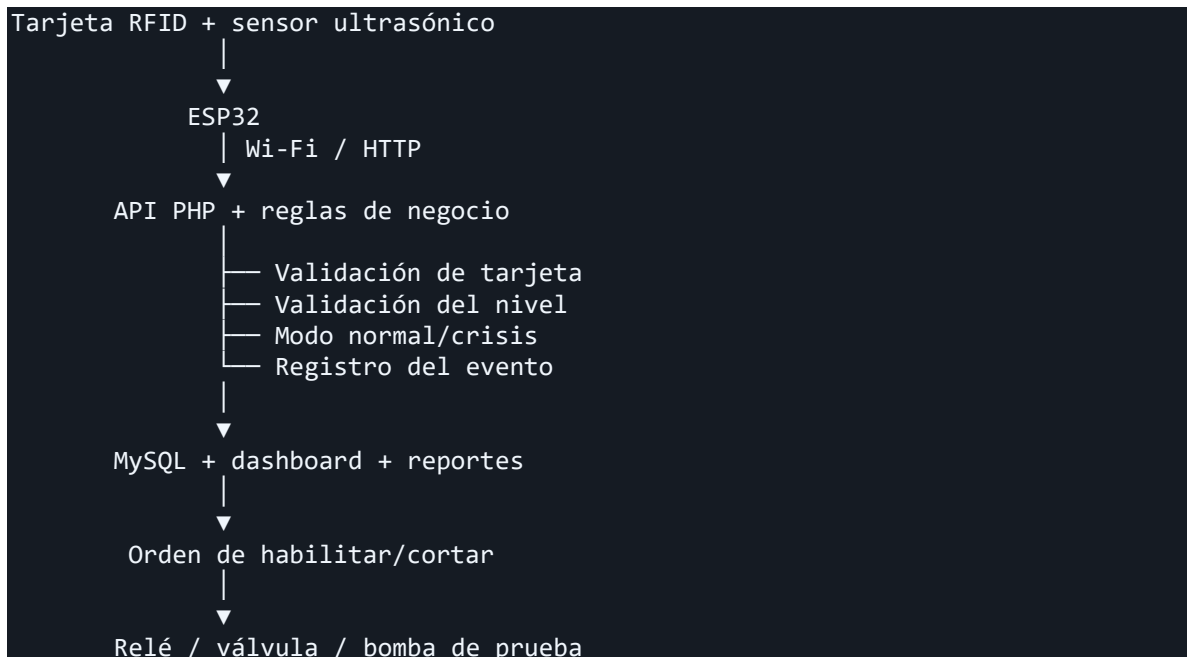
- ESP32 DevKit v1.
- Lector RFID RC522/MFRC522.
- Tarjetas o llaveros MIFARE de prueba.
- Sensor ultrasónico HC-SR04 o JSN-SR04T para estimar distancia y nivel.
- Módulo relé de un canal u otro actuador de corte.

- Contenedor/tanque de prueba, protoboard, fuente regulada y cableado.
- LED, servomotor o mini bomba si forman parte del montaje utilizado.
- Botón de emergencia, si está incorporado en el circuito.

Software y firmware

- Firmware del ESP32 para lectura RFID, adquisición ultrasónica, conexión Wi-Fi y decisión de habilitación/corte.
- Backend PHP con API JSON.
- Autenticación y control de roles.
- Dashboard web.
- Reportes CSV y PDF.
- Base de datos MySQL/MariaDB.

Flujo demostrable



3. Alcance funcional

Funcionalidades implementadas o integradas

- Inicio de sesión.
- Roles de administrador y operador.
- Registro, activación y desactivación de tarjetas RFID.
- Lectura del UID mediante el lector físico o dato de prueba documentado.
- Lecturas del nivel mediante sensor ultrasónico o simulación controlada durante la calibración.
- Envío de telemetría hacia el backend.
- Autorización y rechazo de despachos.
- Modo normal y modo crisis.
- Activación o bloqueo del actuador de prueba.
- Registro histórico de lecturas y despachos.
- Dashboard de nivel, volumen, modo y consumo.
- Reportes CSV y PDF.

Límites de esta versión

- El sensor ultrasónico estima el volumen a partir de distancia, calibración y capacidad del recipiente; no mide litros directamente.
- El montaje es de laboratorio y no debe conectarse a combustible real sin evaluación técnica y normativa especializada.
- La precisión, latencia y autocorte deben respaldarse con mediciones registradas, no solamente con el funcionamiento visual.
- MQTT, si se utiliza, debe documentarse con el broker, tópico, formato del mensaje y evidencia de recepción. El backend PHP revisado utiliza endpoints HTTP; no debe afirmarse MQTT como implementado si no existe código o evidencia de esa integración.

4. Requisitos

Hardware

- ESP32 DevKit v1.
- RC522/MFRC522 y tarjetas MIFARE.
- HC-SR04 o JSN-SR04T.
- Relé o actuador de prueba.
- Fuente regulada de 3.3 V/5 V según el módulo.
- Computador con XAMPP o servidor web PHP.

Software

- PHP 8.x con pdo y pdo_mysql.
- MySQL 8 o MariaDB.
- Apache/XAMPP o servidor integrado de PHP.
- Arduino IDE o PlatformIO para el firmware.
- Librerías de firmware utilizadas por el equipo: WiFi, HTTPClient, ArduinoJson, SPI, MFRC522 y las específicas del sensor.
- curl o Postman para pruebas de API.

5. Instalación de la base de datos

El esquema se encuentra en:

```
database/schema.sql
```

Crear la base de datos:

```
mysql -u root -p -e "CREATE DATABASE controlfuel CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"  
mysql -u root -p controlfuel < database/schema.sql
```

El esquema crea:

Tabla	Propósito
usuarios	Cuentas, roles y último acceso
tarjetas_rfid	UID, titular y estado de las tarjetas
configuracion	Modo, límite de crisis, capacidad y umbral crítico
lecturas_tanque	Histórico de nivel y volumen estimado
despachos	Eventos autorizados/rechazados y motivo

6. Observación importante sobre la configuración

El código PHP revisado consulta `configuracion.litros_normal_max` cuando calcula los litros del modo normal, pero la base de datos proporcionada no contiene esa columna. Antes de ejecutar la prueba T14, el equipo debe elegir una solución y dejarla registrada:

Alternativa recomendada

Agregar la columna al esquema:

```
ALTER TABLE configuracion
ADD COLUMN litros_normal_max DECIMAL(6,2) NOT NULL DEFAULT 10.00
AFTER limite_crisis_litros;
```

Después, actualizar el código y volver a exportar `database/schema.sql` para que el esquema y la aplicación queden sincronizados.

También se recomienda definir un `nivel_critico_pct` distinto de cero para poder probar el rechazo por nivel crítico, por ejemplo:

```
UPDATE configuracion
SET nivel_critico_pct = 10.00,
    litros_normal_max = 10.00
WHERE id = 1;
```

No se debe afirmar que la prueba del modo normal o del nivel crítico está aprobada hasta resolver esta configuración.

7. Configuración del backend

Editar localmente `config/database.php`:

```
define('DB_HOST', 'localhost');
define('DB_NAME', 'controlfuel');
define('DB_USER', 'root');
define('DB_PASS', '');
```

Configurar una API key local en config/api.php y utilizar el mismo valor en el firmware o cliente de prueba. No publicar secretos reales.

8. Crear el usuario administrador

Con Apache y MySQL activos, abrir una sola vez:

```
http://localhost/controlfuel/setup_admin.php
```

Luego:

1. iniciar sesión;
2. cambiar la credencial de demostración;
3. eliminar o restringir setup_admin.php;
4. crear usuarios y tarjetas de prueba desde el sistema.

9. Ejecución del sistema web

XAMPP

- Copiar la carpeta dentro de htdocs.
- Iniciar Apache y MySQL.
- Cargar database/schema.sql.
- Abrir <http://localhost/controlfuel/login.php>.

Servidor integrado

```
php -S 127.0.0.1:8000
```

Abrir:

```
http://127.0.0.1:8000/login.php
```

10. Ejecución del hardware

1. Conectar el RC522 al ESP32 mediante SPI según el cableado documentado por el equipo.
2. Conectar el sensor ultrasónico a los pines definidos en el firmware.
3. Conectar el relé o actuador de prueba respetando niveles lógicos y alimentación.
4. Mantener el actuador en estado seguro/apagado durante el arranque y ante pérdida de Wi-Fi.

5. Cargar el firmware desde Arduino IDE o PlatformIO.
6. Configurar SSID, contraseña Wi-Fi, URL del backend y API key sin publicar esos valores.
7. Encender el ESP32 y verificar en el monitor serial la conexión y el estado seguro.
8. Presentar una tarjeta de prueba.
9. Comprobar la respuesta del backend y el cambio del actuador.
10. Variar el nivel del contenedor y repetir en modo normal y modo crisis.

El repositorio final debe incluir el firmware y un diagrama de pines. Si estos archivos se mantienen fuera del ZIP, deben anexarse mediante el enlace al repositorio.

11. Endpoints HTTP del backend revisado

POST /api/lectura_tanque.php

Encabezados:

```
Content-Type: application/json
X-API-Key: API_KEY_LOCAL
```

Cuerpo:

```
{
  "nivel_pct": 62.4,
  "volumen_litros": 6240
}
```

Respuesta esperada:

```
{
  "ok": true
}
```

POST /api/despacho.php

Cuerpo:

```
{
  "uid_rfid": "A1B2C3D4"
}
```

Respuesta autorizada:

```
{
  "autorizado": true,
  "litros": 10.0,
  "modo": "normal"
}
```

Respuesta rechazada:

```
{
  "autorizado": false,
  "motivo": "Tarjeta no registrada"
}
```

El firmware debe tratar cualquier respuesta no autorizada, error de red o respuesta inválida como condición segura: actuador apagado o bloqueado.

12. Estructura recomendada del repositorio

```
api/
assets/
config/
database/
├─ schema.sql

docs/
├─ arquitectura.png
├─ diagrama-pines.png
├─ modelo-datos.png
└─ pruebas-api.md

firmware/
├─ controlfuel.ino
└─ README.md

includes/
README.md
```

13. Seguridad y alcance académico

- Utilizar datos ficticios y UID de prueba.
- No almacenar API keys, contraseñas Wi-Fi ni credenciales reales en Git.
- No conectar el prototipo a combustible real.
- No presentar el sistema como certificado para atmósferas explosivas.
- Incorporar protección CSRF, gestión segura de secretos y pruebas de pérdida de conexión antes de cualquier uso fuera del laboratorio.

14. Estado de validación

Área	Estado que debe reportarse
Login y roles	Validar con T01-T03
RFID	Validar físicamente con T04 y H01
Sensor ultrasónico	Validar calibración con H02
API de lectura	Validar con T07-T09
Despacho	Validar con T10-T14
Relé/actuador	Validar con H03-H05
Dashboard y reportes	Validar con T15-T17
Persistencia	Verificar en las tablas SQL
Latencia RFID	Medir, no estimar
Precisión volumétrica	Comparar contra volumen de referencia
Autocorte	Medir desde límite hasta apagado